

1 External Security Notes

1.1 Deployment Recommendations

We recommend a simple but safe deployment setup:

- Keep the five storage servers on a private network and do not expose them directly to the public internet.
- Use HTTPS through a reverse proxy if clients are not running on localhost.
- Set a strong shared API token and rotate it if it is leaked.
- Run each server process with limited filesystem permissions so it can only access its own data folder.
- Turn on request logging so failed verification events are easy to trace during debugging and demos.

1.2 Known Limitations

Veri-Store is primarily an *integrity* prototype, not a complete secure storage system. Its verification logic is effective at detecting fragment corruption and many forms of byzantine tampering, but several important security properties remain outside the current design.

- **No confidentiality guarantees.** Fragments are stored and transmitted as plaintext unless the integrating application adds encryption or deploys the system behind TLS. Erasure coding alone does not make the data confidential.
- **The client does not retrieve against a separately trusted FPCC.** During GET, the client currently adopts the first successfully returned `fpcc_json` as the reference value for fragment verification. This means the client proves that fragments are internally consistent with that `fpcc`, but not that the `fpcc` itself came from a trusted source. If at least m malicious servers return the same forged `fpcc` and mutually consistent fragments, the client may reconstruct attacker-chosen data.
- **Block identity is not cryptographically bound into verification.** The current `fpcc` commits to fragment hashes and fingerprints, but not to the application-level `block_id`. As shown by the penetration-testing results (`.\docs\experiments\pentest-attack-matrix.md`), a valid fragment or a full stale response can be replayed under a different block namespace and still pass the low-level fragment checks. This is an accepted limitation of the current prototype and would need to be addressed before claiming stronger object-authenticity guarantees.
- **Authentication is deployment-wide rather than user-specific.** The bearer token protects the service from unauthenticated access, but any party that learns the shared token can read, write, or delete any block whose identifier it knows. There is no per-user authorization, ownership check, or audit identity binding in the current implementation. This is an accepted risk, as the current implementation is meant for a single user, but would need to be mitigated for a multi-user deployment.
- **Security depends on threshold assumptions.** The integrity story assumes that fewer than m servers collude maliciously when the client is relying on server-supplied metadata. The system also assumes the cryptographic primitives behave as expected, in particular the collision resistance of

SHA-256 and the random-oracle-style behavior used in the fingerprint construction. The collision resistance is empirically demonstrated in `.\docs\experiments\collision-probability.md`.

- **Availability and denial-of-service resistance are limited.** Veri-Store can tolerate missing or corrupted fragments up to the reconstruction threshold, but it does not currently provide strong rate limiting (it does have basic rate limiting), storage quotas, admission control, or recovery orchestration suitable for adversarial production environments.

These limitations do not invalidate the prototype's main goal, which is to demonstrate verified integrity checking for erasure-coded fragments. They do, however, define the boundary of the security claims that this document can make: Veri-Store currently offers useful corruption detection and threshold-based integrity checks, but not full end-to-end authenticity, confidentiality, or production-grade access control.

1.3 Integration Considerations